

```
"""
```

```
bootstrap/pa_crosswalk_data.py - Canonical  
Pennsylvania County Crosswalk SSOT
```

```
Single source of truth for the 67-county /  
FIPS / rating-area mapping.
```

```
bootstrap/02_crosswalk.py and  
scripts/bootstrap_data.py both import from  
here instead of each hardcoding their own  
copy – two independently-typed  
67-row tables is exactly the kind of thing  
that silently drifts apart,  
which is the reason this module exists at  
all.
```

```
Provenance: this table matches two  
independently-authored prior versions  
(bootstrap/02_crosswalk.py's original  
RATING_AREAS/COUNTY_FIPS dicts and  
scripts/bootstrap_data.py's COUNTIES_RAW  
string) that agreed on all 67  
counties, and was spot-checked against real  
PA Insurance Department 2026  
rate-filing rating-area listings (e.g.  
Highmark Coverage Advantage: areas  
1 and 4; Geisinger Health Plan: areas  
2,3,5,6,7,9). A third candidate table  
was checked against this one and rejected –  
it disagreed on 57 of 67 rows  
and had a corrupted FIPS/name pairing (see
```

```
conversation history for that
comparison) – so treat any future proposed
replacement of this table the
same way: diff it against this one and
against real filings before
accepting it, not on the strength of
looking well-formatted.
"""
```

```
from typing import Dict, List, TypedDict
```

```
class CountyEntry(TypedDict):
```

```
    fips: str
```

```
    county: str
```

```
    rating_area: int
```

```
PA_COUNTIES_CANONICAL: List[CountyEntry] =
```

```
[
    {"fips": "42001", "county": "Adams",
     "rating_area": 7},
    {"fips": "42003", "county":
     "Allegheny", "rating_area": 4},
    {"fips": "42005", "county":
     "Armstrong", "rating_area": 4},
    {"fips": "42007", "county": "Beaver",
     "rating_area": 4},
    {"fips": "42009", "county": "Bedford",
     "rating_area": 5},
    {"fips": "42011", "county": "Berks",
     "rating_area": 7},
    {"fips": "42013", "county": "Blair",
     "rating_area": 5},
    {"fips": "42015", "county": "Bradford",
```

```
"rating_area": 3},
  {"fips": "42017", "county": "Bucks",
"rating_area": 8},
  {"fips": "42019", "county": "Butler",
"rating_area": 4},
  {"fips": "42021", "county": "Cambria",
"rating_area": 5},
  {"fips": "42023", "county": "Cameron",
"rating_area": 2},
  {"fips": "42025", "county": "Carbon",
"rating_area": 3},
  {"fips": "42027", "county": "Centre",
"rating_area": 6},
  {"fips": "42029", "county": "Chester",
"rating_area": 8},
  {"fips": "42031", "county": "Clarion",
"rating_area": 1},
  {"fips": "42033", "county":
"Clearfield", "rating_area": 5},
  {"fips": "42035", "county": "Clinton",
"rating_area": 3},
  {"fips": "42037", "county": "Columbia",
"rating_area": 6},
  {"fips": "42039", "county": "Crawford",
"rating_area": 1},
  {"fips": "42041", "county":
"Cumberland", "rating_area": 9},
  {"fips": "42043", "county": "Dauphin",
"rating_area": 9},
  {"fips": "42045", "county": "Delaware",
"rating_area": 8},
  {"fips": "42047", "county": "Elk",
"rating_area": 2},
  {"fips": "42049", "county": "Erie",
```

```
"rating_area": 1},
  {"fips": "42051", "county": "Fayette",
"rating_area": 4},
  {"fips": "42053", "county": "Forest",
"rating_area": 1},
  {"fips": "42055", "county": "Franklin",
"rating_area": 9},
  {"fips": "42057", "county": "Fulton",
"rating_area": 9},
  {"fips": "42059", "county": "Greene",
"rating_area": 4},
  {"fips": "42061", "county":
"Huntingdon", "rating_area": 5},
  {"fips": "42063", "county": "Indiana",
"rating_area": 4},
  {"fips": "42065", "county":
"Jefferson", "rating_area": 5},
  {"fips": "42067", "county": "Juniata",
"rating_area": 9},
  {"fips": "42069", "county":
"Lackawanna", "rating_area": 3},
  {"fips": "42071", "county":
"Lancaster", "rating_area": 7},
  {"fips": "42073", "county": "Lawrence",
"rating_area": 4},
  {"fips": "42075", "county": "Lebanon",
"rating_area": 9},
  {"fips": "42077", "county": "Lehigh",
"rating_area": 6},
  {"fips": "42079", "county": "Luzerne",
"rating_area": 3},
  {"fips": "42081", "county": "Lycoming",
"rating_area": 3},
  {"fips": "42083", "county": "McKean",
```

```
"rating_area": 1},
  {"fips": "42085", "county": "Mercer",
"rating_area": 1},
  {"fips": "42087", "county": "Mifflin",
"rating_area": 6},
  {"fips": "42089", "county": "Monroe",
"rating_area": 3},
  {"fips": "42091", "county":
"Montgomery", "rating_area": 8},
  {"fips": "42093", "county": "Montour",
"rating_area": 6},
  {"fips": "42095", "county":
"Northampton", "rating_area": 6},
  {"fips": "42097", "county":
"Northumberland", "rating_area": 6},
  {"fips": "42099", "county": "Perry",
"rating_area": 9},
  {"fips": "42101", "county":
"Philadelphia", "rating_area": 8},
  {"fips": "42103", "county": "Pike",
"rating_area": 3},
  {"fips": "42105", "county": "Potter",
"rating_area": 2},
  {"fips": "42107", "county":
"Schuylkill", "rating_area": 6},
  {"fips": "42109", "county": "Snyder",
"rating_area": 6},
  {"fips": "42111", "county": "Somerset",
"rating_area": 5},
  {"fips": "42113", "county": "Sullivan",
"rating_area": 3},
  {"fips": "42115", "county":
"Susquehanna", "rating_area": 3},
  {"fips": "42117", "county": "Tioga",
```

```

"rating_area": 3},
    {"fips": "42119", "county": "Union",
"rating_area": 6},
    {"fips": "42121", "county": "Venango",
"rating_area": 1},
    {"fips": "42123", "county": "Warren",
"rating_area": 1},
    {"fips": "42125", "county":
"Washington", "rating_area": 4},
    {"fips": "42127", "county": "Wayne",
"rating_area": 3},
    {"fips": "42129", "county":
"Westmoreland", "rating_area": 4},
    {"fips": "42131", "county": "Wyoming",
"rating_area": 3},
    {"fips": "42133", "county": "York",
"rating_area": 7},
]

```

```

# Fail loudly and immediately on import if
the table is ever bad, rather
# than silently deduping-then-asserting-a-
count – a count check alone
# can't tell a corrupted substitution from
a correct table (a wrong FIPS
# swapped in for a missing one keeps the
total at 67 either way).
_fips_seen = set()
_names_seen = set()
for _entry in PA_COUNTIES_CANONICAL:
    if _entry["fips"] in _fips_seen:
        raise AssertionError(f"Duplicate
FIPS in PA_COUNTIES_CANONICAL:
{_entry['fips']}")

```

```

        if _entry["county"] in _names_seen:
            raise AssertionError(f"Duplicate
county name in PA_COUNTIES_CANONICAL:
{_entry['county']}")
        if not (1 <= _entry["rating_area"] <=
9):
            raise AssertionError(f"Rating area
out of range for {_entry['county']}:
{_entry['rating_area']}")
        _fips_seen.add(_entry["fips"])
        _names_seen.add(_entry["county"])
    if len(PA_COUNTIES_CANONICAL) != 67:
        raise AssertionError(f"Expected 67 PA
counties, got
{len(PA_COUNTIES_CANONICAL)}")

def get_flat_crosswalk_dict() -> Dict:
    """Flat-list shape: {"state_fips":
"42", "state_abbr": "PA", "counties":
[...]}. """
    return {
        "state_fips": "42",
        "state_abbr": "PA",
        "counties": [
            {"fips": e["fips"],
"canonical_name": e["county"],
"rating_area": e["rating_area"]}
            for e in PA_COUNTIES_CANONICAL
        ],
    }

def get_area_keyed_crosswalk_dict() ->

```

```

Dict[str, List[Dict[str, str]]]:
    """Area-keyed shape: {"1": [{"county":
"...", "fips": "..."}], "2": [...],
...}."""
    area_map: Dict[str, List[Dict[str,
str]]] = {str(i): [] for i in range(1, 10)}
    for e in PA_COUNTIES_CANONICAL:

area_map[str(e["rating_area"])].append({"co
unt": e["county"], "fips": e["fips"]})
    for area_id in area_map:
        area_map[area_id].sort(key=lambda
x: x["county"])
    return area_map

def get_counties_by_area() -> Dict[int,
List[str]]:
    """{1: ["Clarion", "Crawford", ...], 2:
[...], ...} – for human-readable
reports (e.g.
scripts/bootstrap_data.py's
rating_areas_2026.csv) that
need "which counties are in area N"
without re-deriving it themselves."""
    result: Dict[int, List[str]] = {i: []
for i in range(1, 10)}
    for e in PA_COUNTIES_CANONICAL:

result[e["rating_area"]].append(e["county"]
)
    for area_id in result:

```



```
        result[area_id].sort()  
    return result
```